

SOA2 (11/11/2025)

Nombre:

DNI:

Primer control de teoría

Responde brevemente a las siguientes preguntas justificando tus respuestas. Una respuesta sin justificar no será dada como válida. Todas las preguntas tienen la misma puntuación.

Tenía solamente 16 años cuando Barkus Karant, niño prodigio de los sistemas operativos y originario de TomBridgeTown, Holanda, fue admitido en el grupo de Arquitectura y Sistemas de Andrew S. Tanenbaum en la famosa Universidad de Amsterdam. Tu siempre habías sido el favorito de Andrew, pero la nueva incorporación amenazaba tu reinado. Así que, ni corto ni perezoso, una fría noche de Noviembre de 2005, entraste sigilosamente en la habitación de Barkus y encendiste su ordenador. Pero lo que viste aquella noche te dejó sin habla. Barkus había estado desarrollando un sistema operativo totalmente distinto. Solamente el ver los wrappers que había implementado, te desveló durante un par de noches para entender como funcionaban. El código de wrapper que viste fue el siguiente:

```
01: ENTRY(write)
02:     push %ebp
03:     mov %esp, %ebp
04:     mov $4, %eax
05:     jmp page2
06:     .align 4096
07:     // Next instruction logical address will be aligned to page size
08:     page2:
09:     nop    // This instruction does nothing
10:     .align 4096
11:     pop %ebp
12:     ret
```

Voz en off: la directiva *“align bytes”* fuerza que la dirección lógica de la siguiente instrucción esté alineada al número de *bytes* indicado. Para ello, rellena de bytes 0 el espacio entre la directiva y la siguiente instrucción.

Además, en el espacio de direcciones lógico, correspondiente a la librería libc, donde se encuentra el código de los wrappers, había páginas lógicas cuyo bit de presencia estaba a 0 (no asignadas a páginas físicas). Curiosamente, había tantas de estas páginas lógicas como wrappers.

Y, por último, en ninguno de los ficheros encontraste un handler para las llamadas al sistema. “¿Qué?”, No dabas crédito a lo que veías. Tan rápido como pudiste, hiciste una copia del sistema operativo de Barkus y formateaste su ordenador. “Tiene que ser mio!!! Pero tengo que entender lo que hace porque seguramente me lo preguntarán”, pensaste para tus adentros.

Voz en off: ¡Tranquilo! El mecanismo en sí no es complicado, pero si que tienes que entenderlo bien. Los wrappers descritos están compuestos por 3 páginas lógicas: la primera es hasta la instrucción `jmp page2`, la segunda tiene su bit de presencia a 0 y la tercera contiene las instrucciones `pop %ebp` y `ret`. Lo que está haciendo Barkus es forzar a que se genere una excepción de page fault dentro del wrapper (después de poner el número de servicio en `eax`) cuando se accede a la segunda página lógica, y dentro de la `page_fault_routine` es donde detecta este caso para acabar llamando a las funciones `sys_*`.

Empezaste por los wrappers:

Pregunta 1

¿Cuál es el objetivo de tener wrappers en el sistema?

Portabilidad

Pregunta 2

¿De qué formas se pueden pasar los parámetros de los wrappers al sistema operativo?

Registros, buffer, mixto

Pregunta 3

¿De qué formas se devuelven los resultados desde los wrappers al usuario?

EAX o puntero buffer

Pregunta 4

¿De qué forma está devolviendo el wrapper de Barkus el resultado al usuario?

Por eax, pero no mira si $eax < 0$ para hacer transformación de error

Pregunta 5

¿El wrapper de Barkus asegura la portabilidad del código de usuario?

No... en caso de error, el wrapper devuelve -error en vez de -1 y poner el código de error en errno

Te diste cuenta de que el paso de parámetros del wrapper de Barkus se hace utilizando el registro ebp como puntero a una zona de usuario donde están los parámetros.

Pregunta 6

¿A qué zona de usuario esta apuntado ebp?

A la pila de usuario

Pregunta 7

¿Qué contiene esta zona de usuario?

El frame de activación del wrapper

Pregunta 8

¿Cuántos puntos de entrada tiene el sistema operativo de Barkus?

Int hw + excepciones. Tambien se acepta 1 único punto de entrada si se refieren a llamadas al sistema

En ese momento, recordaste que la excepción de page fault añade un código de error después del contexto hardware y que en el registro cr2 pone la dirección de memoria lógica que ha causado la excepción.

“Creo que lo entiendo todo” te dijiste en voz alta mientras abrías el fichero entry.S para mirar el handler de la excepción de page fault:

```
01: ENTRY(page_fault_handler)
02:     SAVE_ALL
03:     call page_fault_routine
04:     call copy_parameters
05:     movl 0x18(%esp), %eax
06:     call *syscall_table(, 4, %eax)
07:     movl %eax, 0x18(%esp)
08:     call set_return_address
09:     RESTORE_ALL
10:     addl $4, %esp
11:     iret
```

Pregunta 9

¿Cómo obtiene el procesador la dirección de la pila de sistema cuando se produce la excepción de page fault?

Accediendo al campo tss.esp0

SOA2 (11/11/2025)

Nombre:

DNI:

Pregunta 10

Indica el contenido de la pila de sistema antes de ejecutar la instrucción 03 del handler

SS – ESP – PSW – CS – EIP – error – GS – FS – ES – DS – EAX – EBP – ESI – EDI – EDX – ECX – EBX

Pregunta 11

Viendo este handler, ¿el sistema operativo de Barkus es capaz de recuperarse de un acceso incorrecto a memoria?

No, porque si lo hiciese, la `page_fault_routine` volvería al handler y entonces se ejecutaría de forma incorrecta una llamada al sistema.

Pregunta 12

¿Podría ponerse el call `page_fault_routine` entre la línea 01 y 02?

No porque estaríamos perdiendo el valor de algunos registros del procesador antes de guardarlos en la pila de sistema

En ese punto fue cuando te diste cuenta de que Barkus era un genio: para asegurar que el código de alto nivel del sistema siguiese siendo compatible y no se tuviese que modificar, Barkus había creado una función llamada `copy_parameters` que modificaba el contexto software tal y como lo esperan las funciones `sys_*`. La función `copy_parameters` está escrita en C y su cabecera es `void copy_parameters(void)`.

Pregunta 13

Dentro de la función `copy_parameters`, ¿Cómo se obtiene la dirección de usuario donde se encuentran los parámetros de la llamada al sistema?

Se tiene que acceder a la posición de EBP dentro del contexto software.

`Current()->stack[KERNEL_STACK_SIZE - 12] + 8`

Pregunta 14

Dentro de la función `copy_parameters`, ¿Cómo se obtiene la dirección de sistema donde se tienen que copiar los parámetros?

`&Current()->stack[KERNEL_STACK_SIZE - 17]`

Pregunta 15

Indica el Código de `copy_parameters` (no hace falta incluir la comprobación de errores)

`Copy_from_user(Current()->stack[KERNEL_STACK_SIZE - 12] + 8, &Current()->stack[KERNEL_STACK_SIZE - 17], 6 * sizeof(long));`

“Ah! ¡¡Ahora lo entiendo!!”. Dentro del handler, la función `set_return_address`, modifica el contexto hardware para poner la dirección de la siguiente instrucción en modo usuario que se tiene que ejecutar al volver a este modo.

Pregunta 16

Indica como se calcula esta dirección de usuario utilizando el registro `cr2`:

`Address = ((cr2 >> 12) + 1) << 12; // Address = cr2 + PAGE_SIZE;`

Pregunta 17

Indica el código de la función `set_return_address`

`Current()->stack[KERNEL_STACK_SIZE - 5] = ((cr2 >> 12) + 1) << 12;`

Barkus realmente era bueno. Tienes que acabar con él como sea. Aunque aún te queda código por mirar, empiezas a prepararlo todo. Tu risa malvada llega a todos los rincones del campus.

En el fichero sched.c encuentras el truco para conseguir que se provoque una excepción de page fault dentro del wrapper. Barkus ha mapeado la librería libc, donde están los wrappers de las llamadas al sistema, entre las direcciones lógicas LIBC_LOG_INIT y LIBC_LOG_END dentro del proceso Init. Y no solamente eso, sino que sabe que hay 128 wrappers entre estas direcciones y que ningún wrapper comparte paginas lógicas con otro wrapper. Así que lo que hace Barkus es crear un código que desmapee la segunda página de cada uno de los wrappers.

Pregunta 18

Escribe el código necesario, que va dentro de la función donde se crea Init, para desmapear la segunda página lógica de todos los wrappers.

```
for (int i = 0; i < 128; i++)  
{  
    del_ss_pag(PT, (LIBC_LOG_INIT >> 12) + (i * 3) + 1);  
}
```

Pregunta 19

Después del código de la pregunta anterior, ¿Se tiene que hacer flush del TLB ya que se están cambiando los mapeos dentro de la tabla de páginas de Init?

No... init aun no se está ejecutando. Estamos en boot.

Pregunta 20

Sabiendo esto, ¿Cómo se detecta en la page fault routine que el acceso a memoria que ha causado la excepción es incorrecto o se debe a una llamada al sistema?

Mirando si el valor del registro cr2 está entre LIBC_LOG_INIT y LIBC_LOG_END

Desde tu ventana ves como la policía ha entrado en la residencia de estudiantes y está poniéndole unas esposas a Barkus. “Seguramente Andrew ya ha leído mi mail sobre Barkus”. Sonríes. En ese momento, Barkus se gira directamente hacia ti y te mira con una mirada en la que puedes leer: “Me has fastidiado la vida, pero yo te la fastidiaré a ti”. Acabas de condenar a Barkus a una vida de delincuencia en la que muchos estudiantes inocentes de SOA y SO2 sufrirán.

Ya estas acabando. ¿Y cómo hace sys_fork? Porque el mapeo de memoria física ha cambiado.

Pregunta 21

Indica las modificaciones que se tienen que hacer en los bucles de copia de memoria de sys_fork.

No se tiene que hacer ninguna modificación.

Pregunta 22

Indica el contenido la pila de sistema antes de ejecutar la primera instrucción en alto nivel de sys_fork

Ctx hw - error - ctx sw - @ret handler - ebp

Aunque la pila de sistema no sea exactamente igual que la que se crea con un handler de llamadas al sistema como el que se vio en clase, la implementación de sys_fork de Barkus es capaz de calcular de forma independiente al contenido de esta pila, en qué posiciones tiene que poner un ebp fake y la dirección de ret_from_fork.

Pregunta 23

Indica como calcular en qué posición de la pila de sistema se tiene que poner el fake ebp en sys_fork independientemente del contenido y de la estructura de éste pila.

Calculando mediante ebp el offset desde el principio de la pila de sistema del proceso current y diviendo este offset entre el tamaño de cada posición de la pila

Barkus ha puesto como valor de fake ebp el número 3133848266.

SOA2 (11/11/2025)

Nombre:

DNI:

Pregunta 24

¿Es relevante este valor del fake ebp?

No... ya que nunca se utilizará. Aunque te reto a que sepas por qué ese valor.

El día de tu graduación se ve manchado por unos rumores que corren por el campus que apuntan a que se han encontrado unas fotos comprometidas a un estudiante y que ese estudiante ha acabado en una lúgubre prisión del centro de Estados Unidos. Después de tu exposición triunfal en la que has utilizado el sistema operativo diseñado por Barkus, tu tutor, Andrew, te pregunta:

Pregunta 25

¿El EOI del handler de la interrupción de reloj en este sistema operativo puede ir detrás del call a alto nivel?

No. Se pone el EOI antes de la call routine para asegurar que seguirán las interrupciones activas, aunque se haga un task_switch a un proceso nuevo. Para ponerlo después, se tiene que modificar la dirección de retorno al handler en la pila de sistema del proceso hijo para que vuelva a la siguiente instrucción después del call a la clock_routine. Pero esto solo es posible si la pila de sistema de una interrupción hardware y la de las llamadas al sistema es compatible y en este caso no lo es.

Acto seguido, inicias el sistema operativo para demostrar que funciona. Pero tu sonrisa triunfal se convierte en una mueca de sorpresa cuando ves que en la pantalla aparece: "Booting ZeOS by BAKA BAKA". Tus días de gloria se han acabado....

Información del Sistema Operativo

El sistema dispone de las siguientes rutinas:

- **struct task_struct *current():** Retorna la task_struct del proceso actual.
- **int alloc_frame():** Reserva un frame de memoria física.
- **void free_frame (int frame):** Libera un frame de memoria.
- **page_table_entry *get_PT(struct task_struct *t):** Retorna la tabla de páginas del proceso t.
- **page_table_entry *get_DIR(struct task_struct *t):** Retorna el directorio de páginas del proceso t.
- **set_CR3 (page_table_entry * dir):** Sobreescribe el registro CR3 con el nuevo directorio de páginas dir, provoca una invalidación de la TLB.
- **int get_frame (page_table_entry *pt, int logical_page):** Retorna el frame asignado a una página lógica en la tabla de páginas de un proceso determinado.
- **int set_ss_page (page_table_entry *pt, int logical_page, int frame):** Asigna un frame a una página lógica de la tabla de páginas pt.
- **int del_ss_page (page_table_entry *pt, int logical_page):** Borra una entrada de la tabla de páginas.